

Loan Default Prediction using ML

```
In [1]: # Importing necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

```
In [2]: #Loading the dataset
ldp = pd.read_csv("Loan_default_data.csv")
```

LOADING AND UNDERSTANDING DATA

```
In [3]: # Basic Data Understanding
print("SHAPE OF DATA:", ldp.shape)
print("\nFIRST FEW ROWS:")
display(ldp.head())
print("\nDATASET INFORMATION:")
ldp.info()
print("\nDESCRIPTIVE STATISTICS OF THE DATASET:")
display(ldp.describe())
```

SHAPE OF DATA: (257603, 18)

FIRST FEW ROWS:

	LoanID	Age	Income	LoanAmount	CreditScore	MonthsEmployed	NumCreditLi
0	I38PQUQS96	56.0	85994.0	50587.0	520.0	80.0	
1	HPSK72WA7R	69.0	50432.0	124440.0	458.0	15.0	
2	C1OZ6DPJ8Y	46.0	84208.0	129188.0	451.0	26.0	
3	V2KKSFM3UN	32.0	31713.0	44799.0	743.0	0.0	
4	EY08JDHTZP	60.0	20437.0	9139.0	633.0	8.0	



DATASET INFORMATION:

<class 'pandas.DataFrame'>

RangeIndex: 257603 entries, 0 to 257602

Data columns (total 18 columns):

#	Column	Non-Null Count	Dtype
0	LoanID	257603 non-null	str
1	Age	255561 non-null	float64
2	Income	255557 non-null	float64
3	LoanAmount	255549 non-null	float64
4	CreditScore	255697 non-null	float64
5	MonthsEmployed	255481 non-null	float64
6	NumCreditLines	257603 non-null	int64
7	InterestRate	255731 non-null	float64
8	LoanTerm	257603 non-null	int64
9	DTIRatio	255563 non-null	float64
10	Education	257603 non-null	str
11	EmploymentType	257603 non-null	str
12	MaritalStatus	257603 non-null	str
13	HasMortgage	257603 non-null	str
14	HasDependents	257603 non-null	str
15	LoanPurpose	257603 non-null	str
16	HasCoSigner	257603 non-null	str
17	Default	257603 non-null	int64

dtypes: float64(7), int64(3), str(8)

memory usage: 35.4 MB

DESCRIPTIVE STATISTICS OF THE DATASET:

	Age	Income	LoanAmount	CreditScore	MonthsEmployed	NumCreditLines
count	255561.000000	2.555570e+05	2.555490e+05	255697.000000	255481.000000	257603.000000
mean	43.502119	8.472380e+04	1.306194e+05	575.011830	59.553239	2.576030e+05
std	14.993387	6.278980e+04	1.022895e+05	167.665449	34.640378	1.022895e+05
min	18.000000	-1.499840e+04	-2.499330e+04	1.000000	0.000000	1.000000
25%	31.000000	4.863400e+04	6.574900e+04	436.000000	30.000000	1.000000
50%	43.000000	8.246200e+04	1.275040e+05	574.000000	60.000000	1.000000
75%	56.000000	1.164330e+05	1.892910e+05	713.000000	90.000000	1.000000
max	69.000000	1.498900e+06	2.488630e+06	1500.000000	119.000000	1.000000

DATA CLEANING

```
In [4]: # CHECKING FOR DUPLICATE
print("\nNUMBER OF DUPLICATE ROWS IN THE DATASET:", ldp.duplicated().sum())
# VIEWING DUPLICATE ROWS
print("\nDUPLICATE ROWS IN THE DATASET:")
print(ldp[ldp.duplicated()])
#DROPPING DUPLICATE ROWS
print(ldp.drop_duplicates(inplace=True))
print("\nSHAPE OF DATA AFTER DROPPING DUPLICATES:", ldp.shape)
```

NUMBER OF DUPLICATE ROWS IN THE DATASET: 1935

DUPLICATE ROWS IN THE DATASET:

	LoanID	Age	Income	LoanAmount	CreditScore	MonthsEmployed	\
255348	6QHQ3DY1JW	40.0	24867.0	114210.0	341.0	58.0	
255349	3WU3D6WWCM	45.0	92461.0	209838.0	309.0	79.0	
255350	PLUW4XQ9KG	33.0	54862.0	139152.0	742.0	113.0	
255351	Q7FHSAXA08	66.0	62385.0	186300.0	688.0	82.0	
255352	EEZ5ZJ8SUT	52.0	147784.0	38490.0	520.0	32.0	
...	...	...	...	...	...	...	
257594	A3DM1I14B3	22.0	88452.0	193666.0	NaN	64.0	
257597	QBGLM7IWT2	21.0	NaN	128956.0	802.0	99.0	
257598	OLGHHGRIWI	28.0	32555.0	13259.0	718.0	44.0	
257599	AI3K3VBW60	43.0	54427.0	109635.0	484.0	0.0	
257600	6A2HAMSVAQ	38.0	62805.0	248384.0	597.0	17.0	

	NumCreditLines	InterestRate	LoanTerm	DTIRatio	Education	\
255348	2	15.46	48	0.67	Bachelor's	
255349	1	21.42	24	0.66	High School	
255350	1	6.59	48	0.10	Master's	
255351	3	13.49	48	NaN	Master's	
255352	4	19.16	12	0.52	Master's	
...	...	...	...	...	...	
257594	2	4.75	24	0.76	Bachelor's	
257597	1	10.94	12	0.19	PhD	
257598	1	17.88	60	0.82	High School	
257599	1	17.20	48	0.74	High School	
257600	3	22.31	36	0.63	Master's	

	EmploymentType	MaritalStatus	HasMortgage	HasDependents	LoanPurpose	\
255348	Full-time	Married	Yes	No	Other	
255349	Self-employed	Divorced	Yes	No	Business	
255350	Self-employed	Divorced	Yes	Yes	Other	
255351	Part-time	Divorced	No	No	Other	
255352	Self-employed	Single	Yes	Yes	Business	
...	...	...	...	...	...	
257594	Part-time	Married	Yes	No	Education	
257597	Self-employed	Married	Yes	No	Other	
257598	Self-employed	Married	No	Yes	Business	
257599	Self-employed	Married	No	Yes	Home	
257600	Unemployed	Single	Yes	No	Home	

	HasCoSigner	Default
255348	Yes	0
255349	No	0
255350	Yes	0
255351	Yes	0
255352	No	0
...	...	...
257594	No	0
257597	Yes	0
257598	No	0
257599	No	0
257600	No	0

[1935 rows x 18 columns]

None

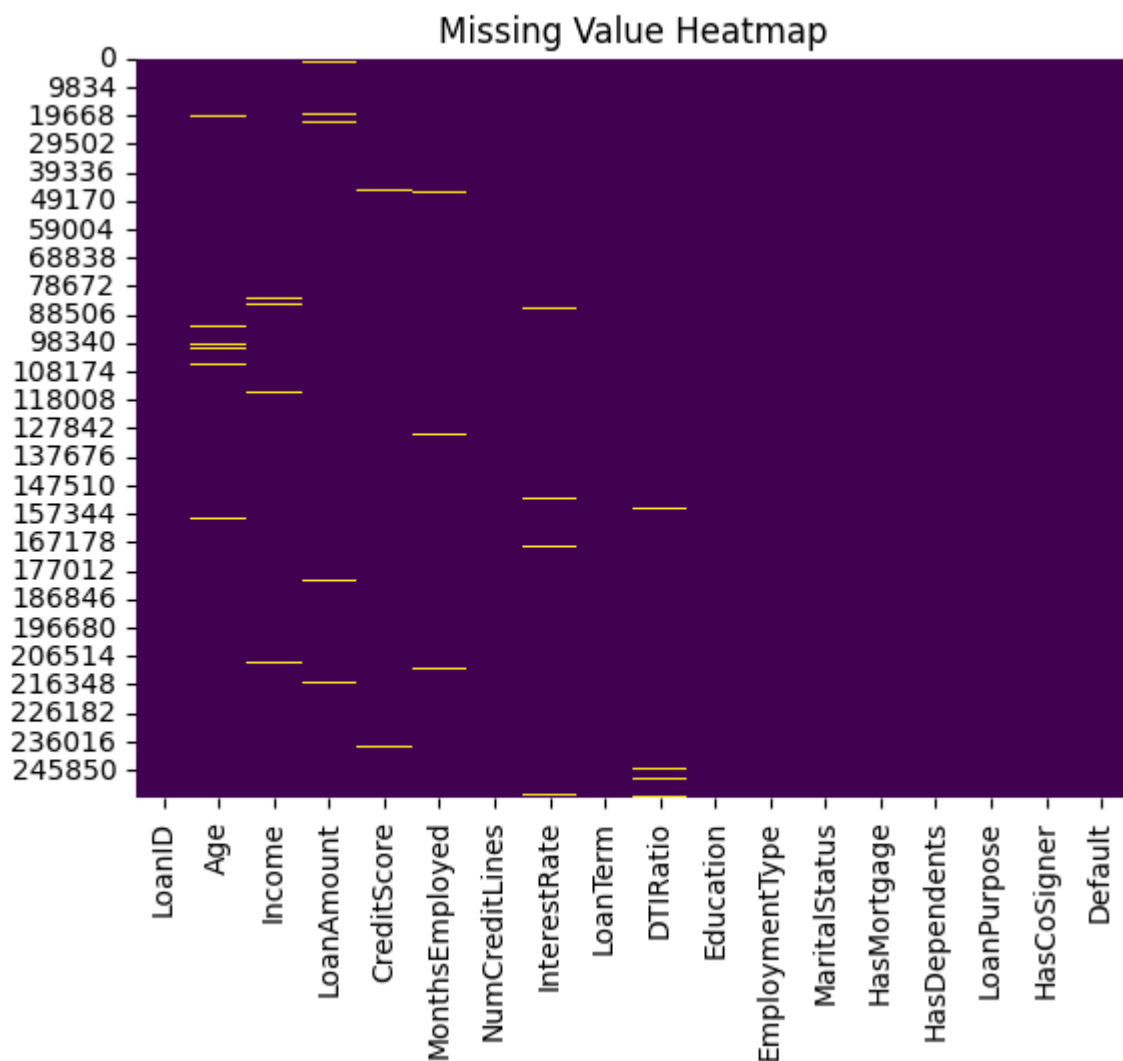
SHAPE OF DATA AFTER DROPPING DUPLICATES: (255668, 18)

```
In [5]: # MISSING VALUES ANALYSIS
print("Missing Values:")
missing_df = pd.DataFrame({
    'Columns': ldp.columns,
    'Missing_Count': ldp.isnull().sum(),
    'Missing_Percentage': (ldp.isnull().sum() / len(ldp) * 100).round(2)
})
print(missing_df[missing_df['Missing_Count'] > 0])
```

Missing Values:

	Columns	Missing_Count	Missing_Percentage
Age	Age	1968	0.77
Income	Income	1947	0.76
LoanAmount	LoanAmount	1974	0.77
CreditScore	CreditScore	1817	0.71
MonthsEmployed	MonthsEmployed	2054	0.80
InterestRate	InterestRate	1793	0.70
DTIRatio	DTIRatio	1953	0.76

```
In [6]: # Visualize missing values
sns.heatmap(ldp.isnull(), cbar=False, cmap='viridis')
plt.title('Missing Value Heatmap')
plt.show()
```



```
In [7]: # HANDLING MISSING VALUES
# We will drop rows with missing values because the percentage of missing values
ldp.dropna(subset=[
```

```
'Age', 'Income', 'LoanAmount', 'CreditScore', 'MonthsEmployed', 'InterestRate', 'D
'], inplace=True)
```

```
# Dataset shape after handling missing values
print(f"Dataset shape after handling missing values: {ldp.shape}")
```

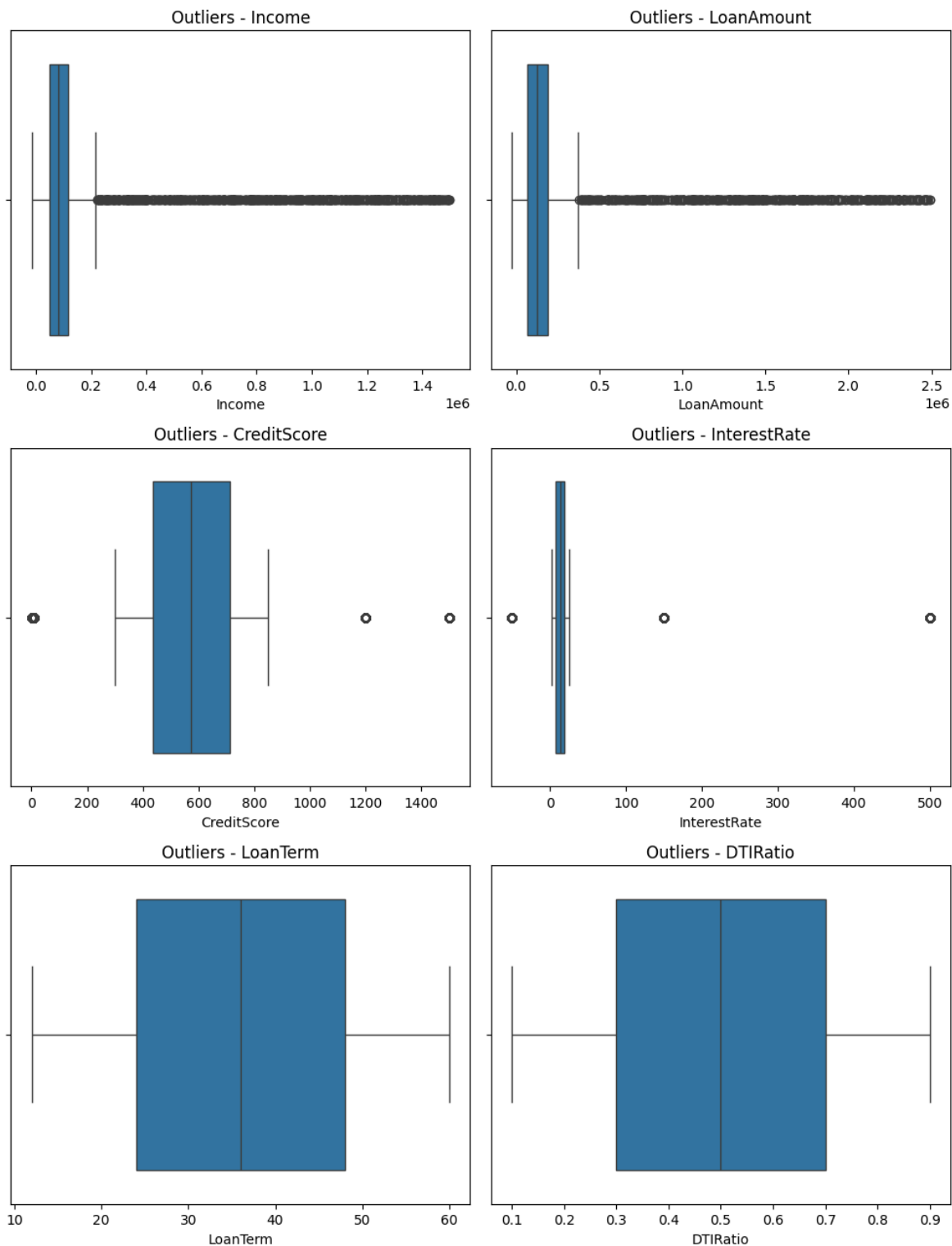
Dataset shape after handling missing values: (242162, 18)

```
In [12]: # Outlier Detection using IQR
# find outlier count per column
columns_to_check = ['Age', 'Income', 'LoanAmount', 'CreditScore',
                    'MonthsEmployed', 'NumCreditLines', 'InterestRate', 'LoanTerm',
                    'DTIRatio']
for col in columns_to_check:
    Q1 = ldp[col].quantile(0.25)
    Q3 = ldp[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = ldp[(ldp[col] < lower) | (ldp[col] > upper)]
    print(f'{col}: {len(outliers)} outliers')
```

Age: 0 outliers
Income: 610 outliers
LoanAmount: 501 outliers
CreditScore: 1183 outliers
MonthsEmployed: 0 outliers
NumCreditLines: 0 outliers
InterestRate: 1295 outliers
LoanTerm: 0 outliers
DTIRatio: 0 outliers

```
In [13]: # VISUALIZE OUTLIERS USING BOX PLOTS
cols_has_outlier = ['Income', 'LoanAmount', 'CreditScore',
                    'InterestRate', 'LoanTerm', 'DTIRatio']

fig, axes = plt.subplots(3, 2, figsize=(10,13))
axes = axes.flatten()
for i, col in enumerate(cols_has_outlier):
    sns.boxplot(x=ldp[col], ax=axes[i])
    axes[i].set_title(f'Outliers - {col}')
plt.tight_layout()
plt.show()
```



```
In [19]: # Dropping outliers based on the IQR bounds calculated previously
for col in cols_has_outlier:
    Q1 = ldp[col].quantile(0.25)
    Q3 = ldp[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    # Keeping only the data within the bounds
    ldp = ldp[(ldp[col] >= lower) & (ldp[col] <= upper)]

print(f"Dataset shape after dropping outliers: {ldp.shape}")
```

Dataset shape after dropping outliers: (238573, 18)

UNIVARIATE ANALYSIS

```
In [58]: # Numerical Columns Distribution Analysis

numeric_cols = [
    'Age', 'Income', 'LoanAmount', 'CreditScore',
    'MonthsEmployed', 'NumCreditLines',
    'InterestRate', 'LoanTerm', 'DTIRatio'
]

fig, axes = plt.subplots(5, 2, figsize=(12, 18))
axes = axes.flatten()

for i, col in enumerate(numeric_cols):

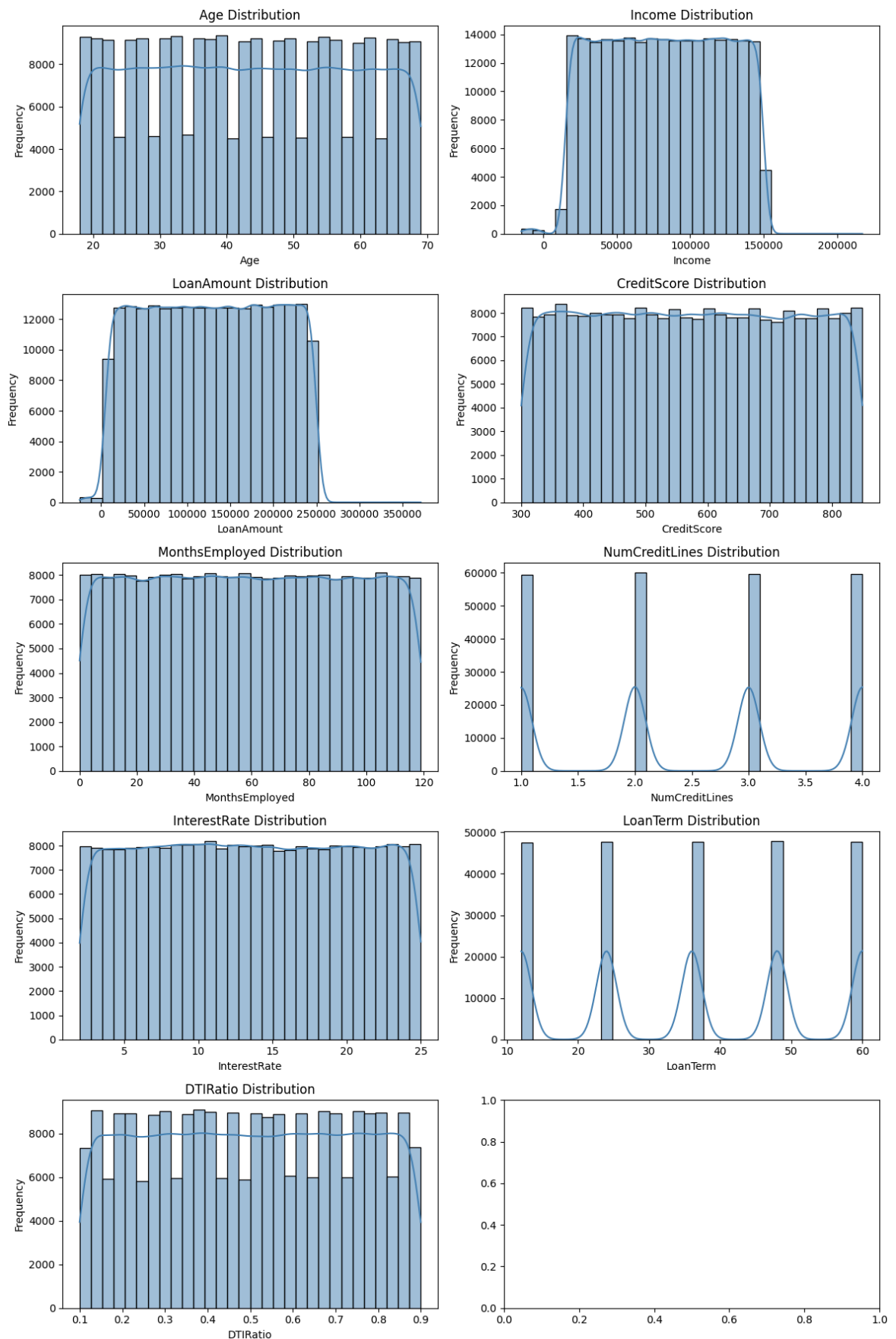
    # Histogram with KDE Curve
    sns.histplot(
        ldp[col],
        kde=True,
        bins=30,
        color='steelblue',
        edgecolor='black',
        ax=axes[i],
        line_kws={'color': 'Black'}
    )

    axes[i].set_title(f'{col} Distribution')
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Frequency')

plt.tight_layout()
plt.show()

# Statistical Summary
for col in numeric_cols:

    print(f"\n{col}")
    print(f"  Mean   : {ldp[col].mean():.3f}")
    print(f"  Median : {ldp[col].median():.3f}")
    print(f"  Std    : {ldp[col].std():.3f}")
    print(f"  Skew   : {ldp[col].skew():.3f}")
    print(f"  Kurt   : {ldp[col].kurt():.3f}")
    print("-" * 35)
```



Age

Mean : 43.432
Median : 43.000
Std : 15.002
Skew : 0.006
Kurt : -1.200

Income

Mean : 82192.120
Median : 82226.000
Std : 39243.921
Skew : -0.013
Kurt : -1.170

LoanAmount

Mean : 127398.785
Median : 127471.000
Std : 71126.860
Skew : -0.007
Kurt : -1.191

CreditScore

Mean : 573.938
Median : 573.000
Std : 158.920
Skew : 0.007
Kurt : -1.201

MonthsEmployed

Mean : 59.447
Median : 59.000
Std : 34.647
Skew : 0.001
Kurt : -1.200

NumCreditLines

Mean : 2.501
Median : 2.000
Std : 1.117
Skew : 0.000
Kurt : -1.358

InterestRate

Mean : 13.509
Median : 13.480
Std : 6.639
Skew : 0.002
Kurt : -1.198

LoanTerm

Mean : 36.025
Median : 36.000
Std : 16.970

```
Skew    : -0.002
Kurt     : -1.300
```

DTIRatio

```
Mean     : 0.500
Median   : 0.500
Std      : 0.231
Skew     : -0.002
Kurt     : -1.200
```

```
In [56]: # Categorical Columns Univariate Analysis
categorical_cols = [
    'Education', 'EmploymentType', 'MaritalStatus', 'HasMortgage',
    'HasDependents', 'LoanPurpose', 'HasCoSigner']

fig, axes = plt.subplots(4, 2, figsize=(10, 15))
axes = axes.flatten()

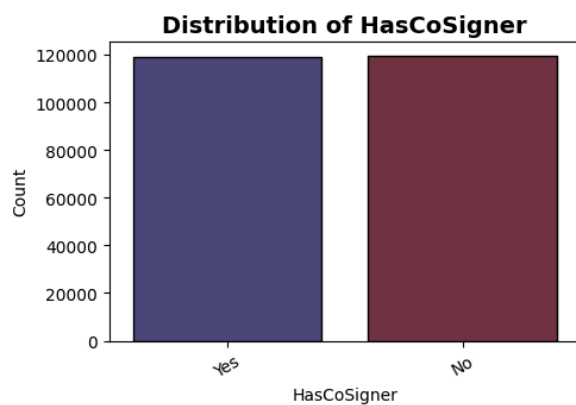
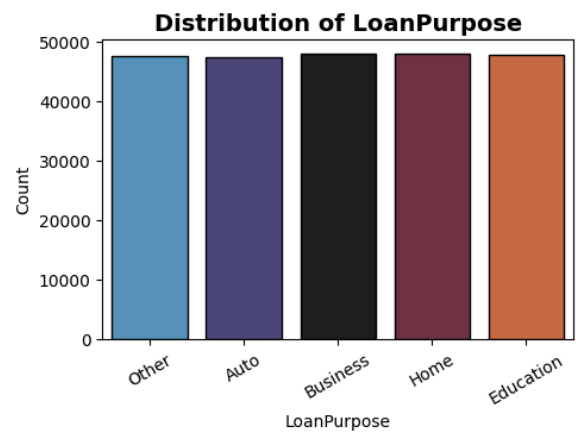
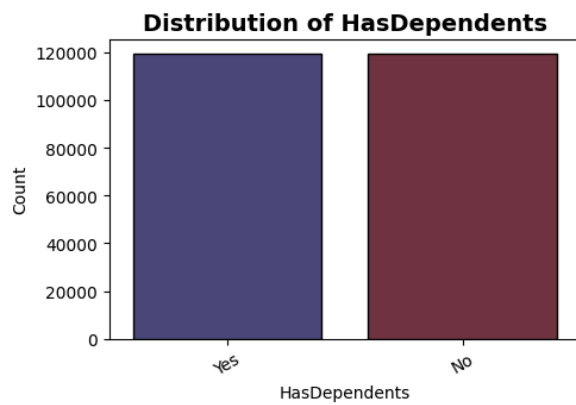
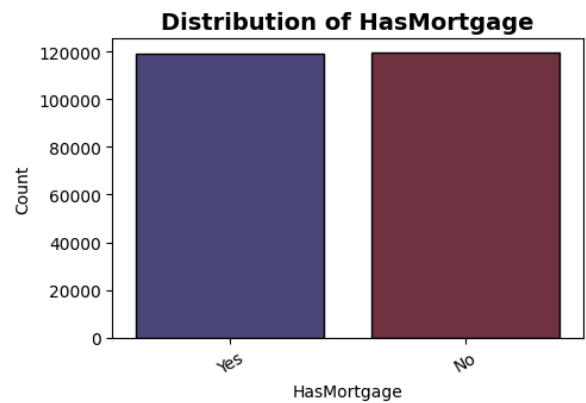
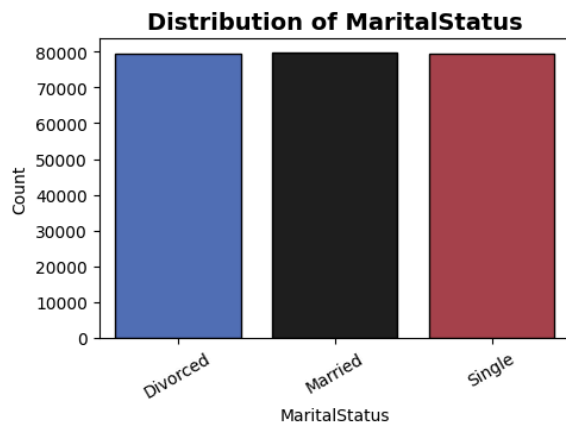
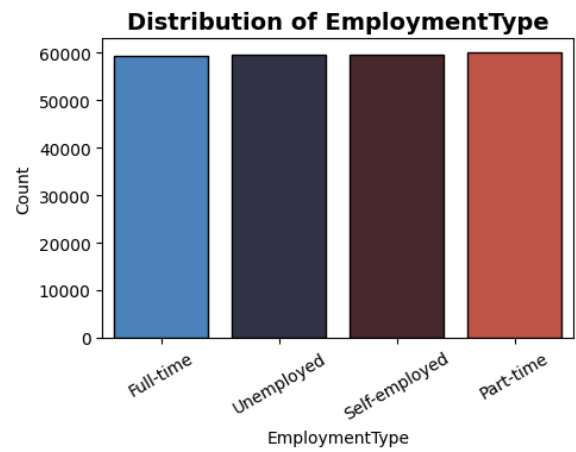
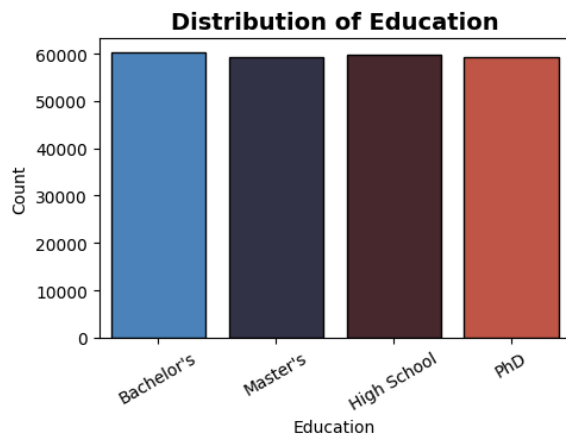
for i, col in enumerate(categorical_cols):
    sns.countplot(data=ldp, x=col, ax=axes[i], palette="icefire", edgecolor='b')

    # Adding Labels and formatting
    axes[i].set_title(f'Distribution of {col}', fontsize=14, fontweight='bold')
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Count')
    axes[i].tick_params(axis='x', rotation=30) # Rotate for better readability

# Remove the 8th (empty) subplot
fig.delaxes(axes[7])

plt.tight_layout()
plt.show()

# Displaying text-based frequency distribution
for col in categorical_cols:
    print(f"\n--- Frequency Table for {col} ---")
    print(ldp[col].value_counts())
```



--- Frequency Table for Education ---

Education

Bachelor's 60187

High School 59841

Master's 59292

PhD 59253

Name: count, dtype: int64

--- Frequency Table for EmploymentType ---

EmploymentType

Part-time 60013

Unemployed 59711

Self-employed 59538

Full-time 59311

Name: count, dtype: int64

--- Frequency Table for MaritalStatus ---

MaritalStatus

Married 79729

Divorced 79429

Single 79415

Name: count, dtype: int64

--- Frequency Table for HasMortgage ---

HasMortgage

No 119452

Yes 119121

Name: count, dtype: int64

--- Frequency Table for HasDependents ---

HasDependents

No 119377

Yes 119196

Name: count, dtype: int64

--- Frequency Table for LoanPurpose ---

LoanPurpose

Business 47930

Home 47917

Education 47698

Other 47588

Auto 47440

Name: count, dtype: int64

--- Frequency Table for HasCoSigner ---

HasCoSigner

No 119507

Yes 119066

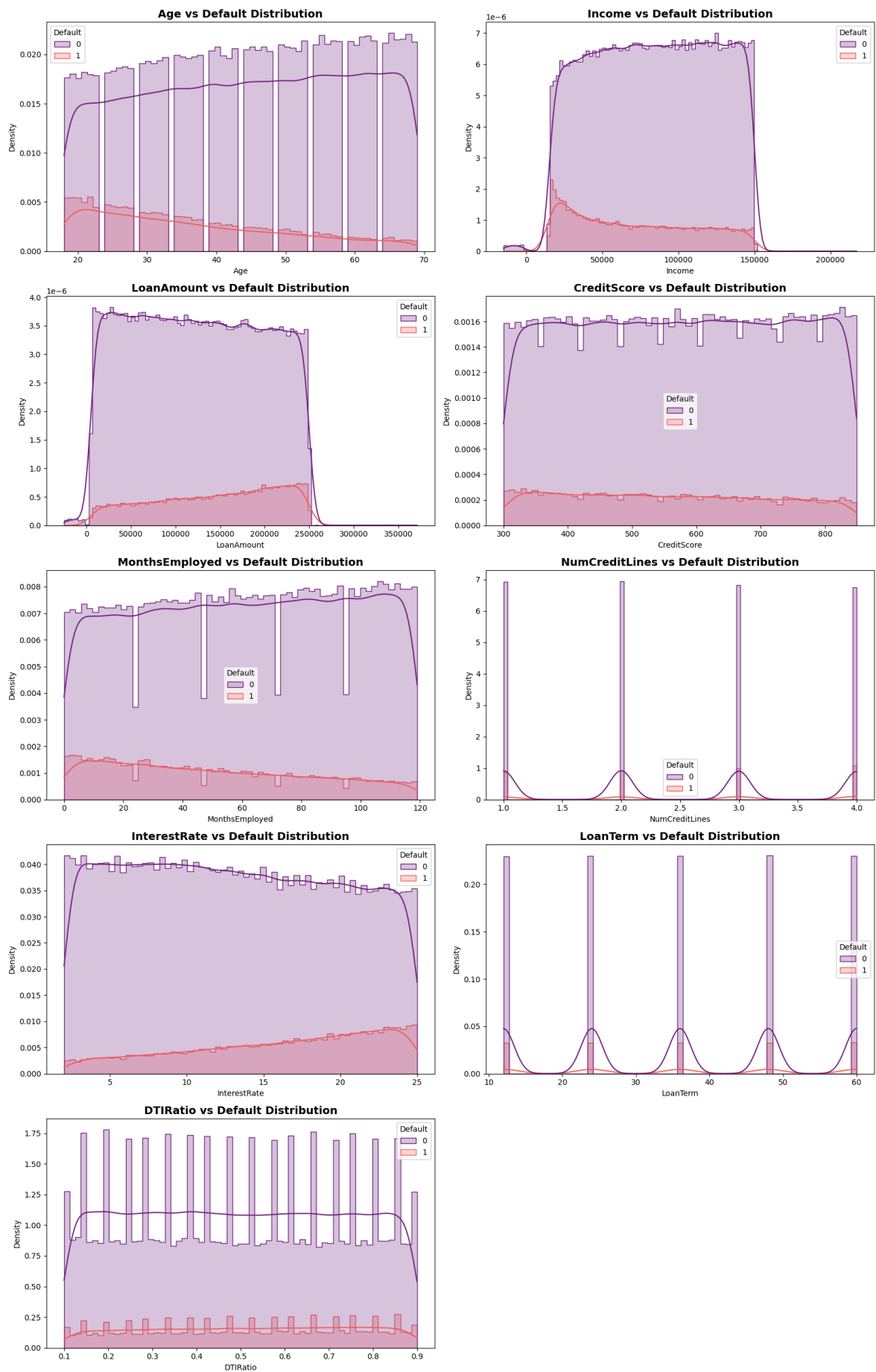
Name: count, dtype: int64

Bivariate Analysis

```
In [67]: # Fix: Numerical vs Default Bivariate Analysis (Histplot with KDE)
numeric_cols = ['Age', 'Income', 'LoanAmount', 'CreditScore', 'MonthsEmployed',

# 1. Create a single figure with enough subplots for all columns
fig, axes = plt.subplots(nrows=5, ncols=2, figsize=(16, 25))
axes = axes.flatten() # Convert 2D array to 1D for easy looping
```

```
for i, col in enumerate(numeric_cols):  
    # 2. Use histplot with hue to compare distributions  
    # stat='density' allows fair comparison if classes are imbalanced  
    sns.histplot(data=ldp, x=col, hue='Default', kde=True,  
                 ax=axes[i], palette='magma', element='step', stat='density')  
  
    # 3. Formatting  
    axes[i].set_title(f'{col} vs Default Distribution', fontsize=14, fontweight=  
axes[i].set_xlabel(col)  
axes[i].set_ylabel('Density')  
  
# 4. Remove the empty 10th subplot  
fig.delaxes(axes[9])  
  
plt.tight_layout()  
plt.show()  
  
# 5. Correlation Check  
print("Numerical Correlation with Default:")  
print(ldp[numeric_cols + ['Default']].corr()['Default'].sort_values(ascending=Fa
```



Numerical Correlation with Default:

```
Default      1.000000
InterestRate 0.135450
LoanAmount   0.090662
NumCreditLines 0.029497
DTIRatio     0.019756
LoanTerm     0.000594
CreditScore -0.034735
Income       -0.099335
MonthsEmployed -0.100164
Age          -0.172589
Name: Default, dtype: float64
```

```
In [68]: # Categorical Columns vs Default

fig, axes = plt.subplots(4, 2, figsize=(16, 22))
axes = axes.flatten()

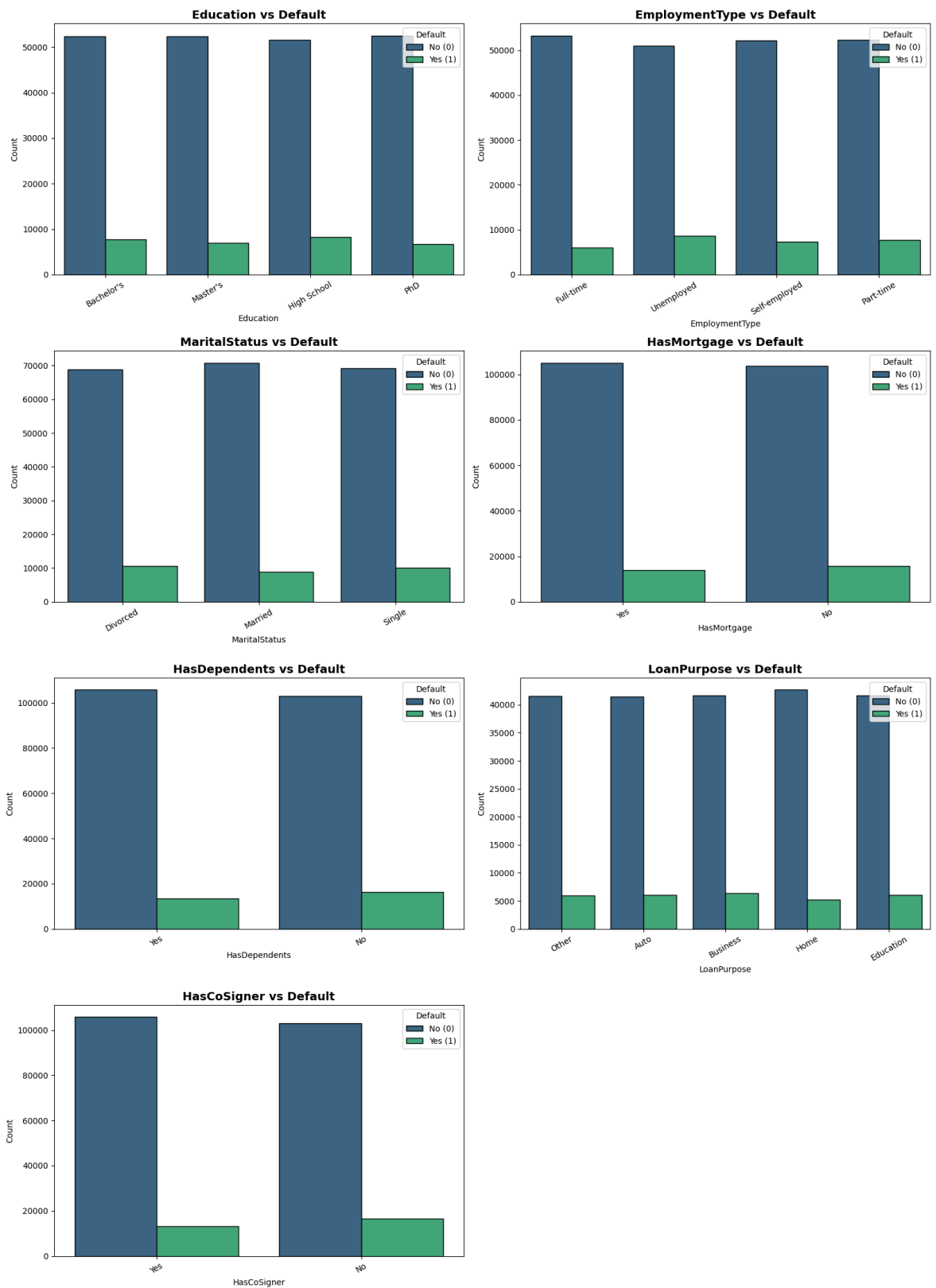
for i, col in enumerate(categorical_cols):
    # Plotting Countplot with 'Default' as the hue to see the split
    sns.countplot(data=ldp, x=col, hue='Default', ax=axes[i], palette='viridis',

    axes[i].set_title(f'{col} vs Default', fontsize=14, fontweight='bold')
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Count')
    axes[i].legend(title='Default', labels=['No (0)', 'Yes (1)'])
    axes[i].tick_params(axis='x', rotation=30)

fig.delaxes(axes[7])

plt.tight_layout()
plt.show()

# Calculating the Default Rate (%) per category for deeper insight
print("--- Default Rate (%) by Category ---")
for col in categorical_cols:
    default_rate = ldp.groupby(col)['Default'].mean() * 100
    print(f"\n{col}:")
    print(default_rate.sort_values(ascending=False).round(2))
```



--- Default Rate (%) by Category ---

Education:

Education

High School 13.75

Bachelor's 12.94

Master's 11.65

PhD 11.35

Name: Default, dtype: float64

EmploymentType:

EmploymentType

Unemployed 14.49

Part-time 12.79

Self-employed 12.26

Full-time 10.16

Name: Default, dtype: float64

MaritalStatus:

MaritalStatus

Divorced 13.42

Single 12.75

Married 11.12

Name: Default, dtype: float64

HasMortgage:

HasMortgage

No 13.19

Yes 11.66

Name: Default, dtype: float64

HasDependents:

HasDependents

No 13.60

Yes 11.26

Name: Default, dtype: float64

LoanPurpose:

LoanPurpose

Business 13.19

Auto 12.73

Education 12.66

Other 12.61

Home 10.95

Name: Default, dtype: float64

HasCoSigner:

HasCoSigner

No 13.74

Yes 11.11

Name: Default, dtype: float64

Multivariate Analysis

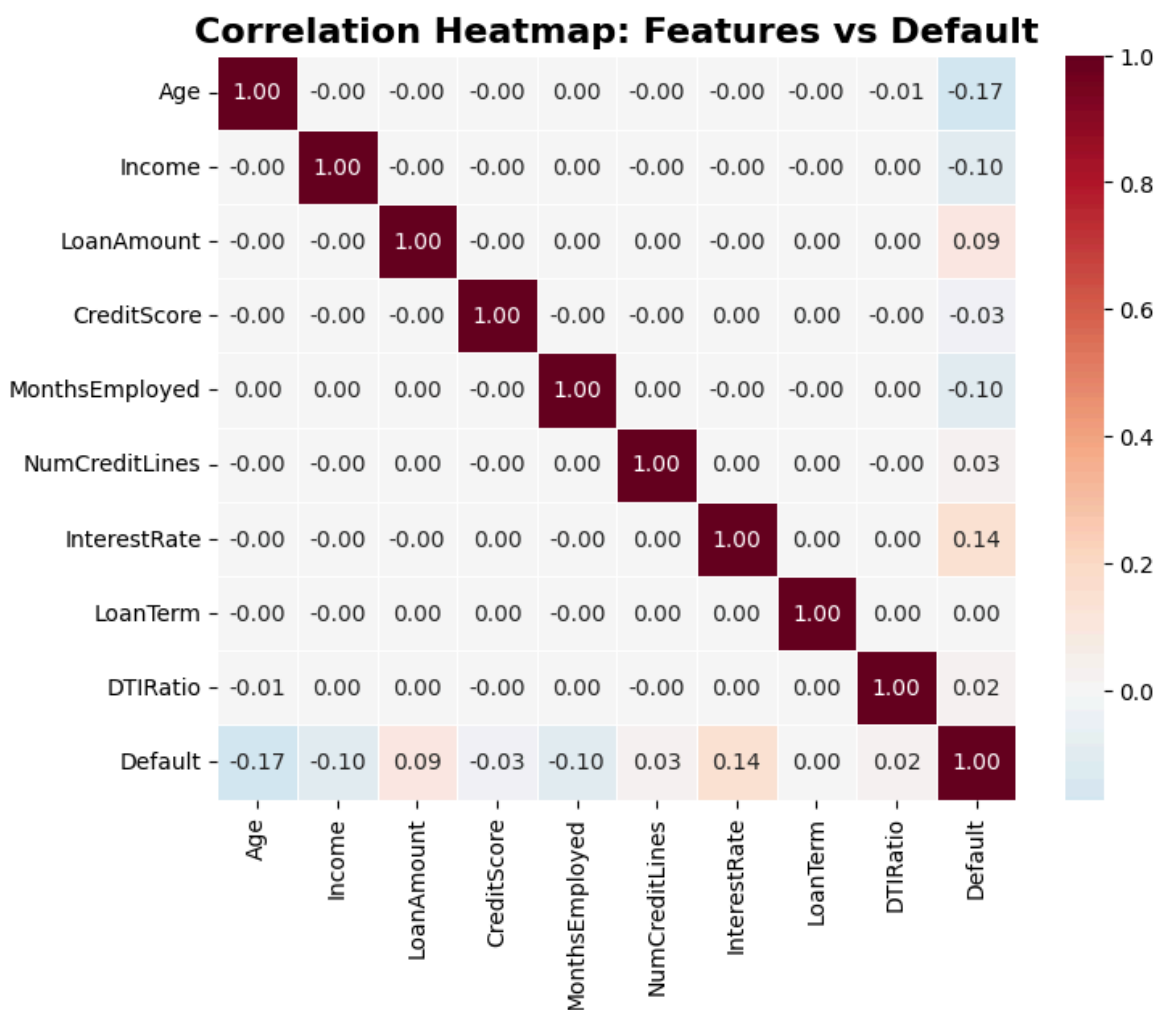
```
In [74]: # Correlation Analysis
plt.figure(figsize=(8, 6))

# 1. Calculate correlation for all numerical columns including the target 'Default'
correlation_matrix = ldp[numeric_cols + ['Default']].corr()
```

```
# 2. Plotting the Heatmap
sns.heatmap(correlation_matrix,
             annot=True,
             cmap='RdBu_r',
             center=0,
             fmt='.2f',
             linewidths=0.5)

plt.title('Correlation Heatmap: Features vs Default', fontsize=16, fontweight='b')
plt.show()

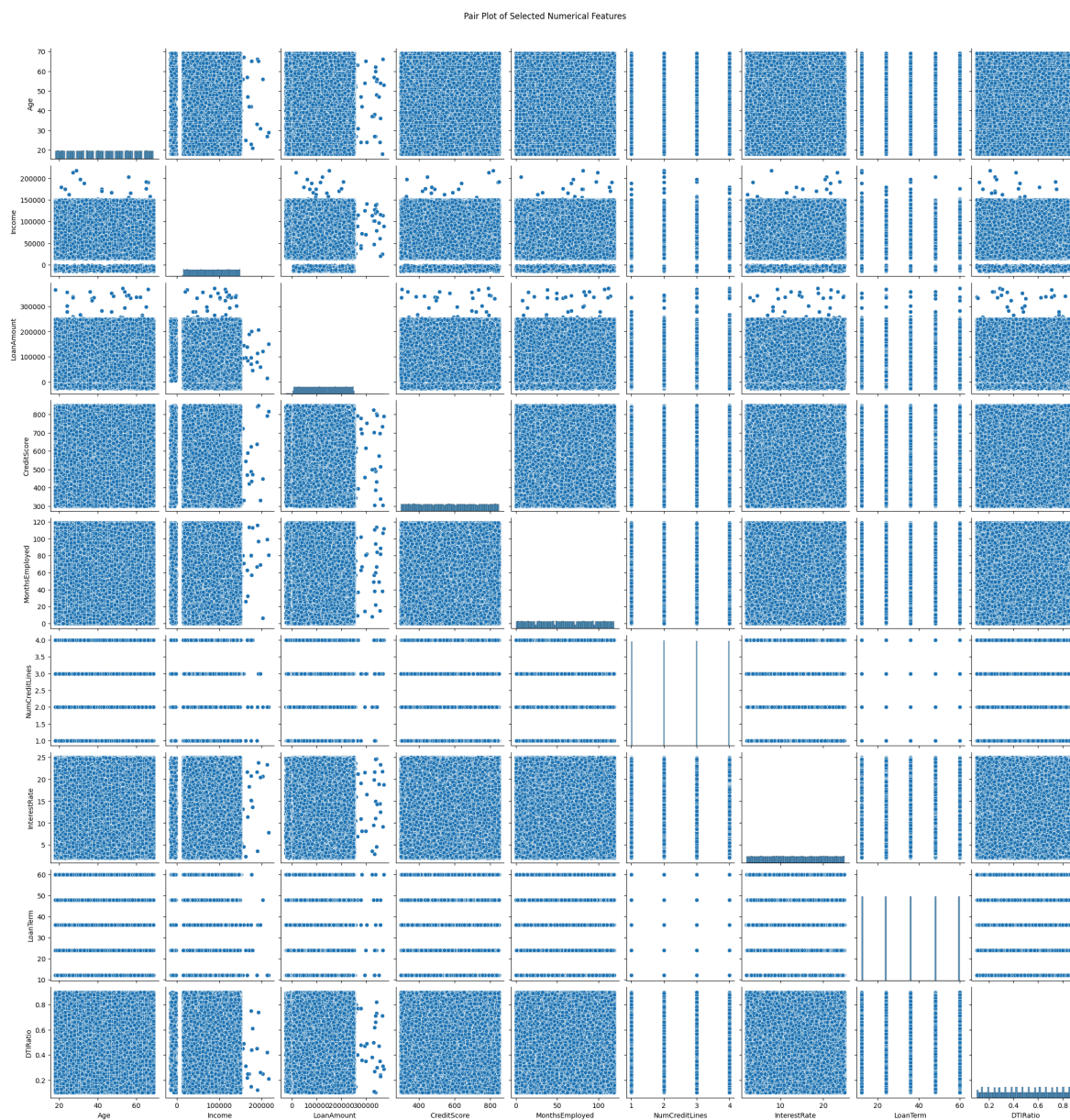
# 3. Targeted Correlation: See exactly what affects Default the most
print("--- Features Ranked by Correlation with Default ---")
correlations = ldp[numeric_cols + ['Default']].corr()['Default'].sort_values(ascending=True)
print(correlations)
```



--- Features Ranked by Correlation with Default ---

```
Default      1.000000
InterestRate 0.135450
LoanAmount   0.090662
NumCreditLines 0.029497
DTIRatio     0.019756
LoanTerm     0.000594
CreditScore  -0.034735
Income       -0.099335
MonthsEmployed -0.100164
Age          -0.172589
Name: Default, dtype: float64
```

```
In [73]: # Pair Plot of Numerical Features
sns.pairplot(ldp[numeric_cols])
plt.suptitle('Pair Plot of Selected Numerical Features', y=1.02)
plt.show()
```



```
In [ ]:
```